AI ASSISTANT CODING - ASSIGNMENT-21.3

Name: Valaboju Harshavardhanu
Roll No: 2303A52354 **Batch:** 41

Task 1: AI-Assisted Pair Programming

Initial prompt:

Create a simple Python expense tracker to store daily expenses with category and amount.

Code:

```
expenses = []



while True:

    name = input("Enter expense name: ")

    amount = float(input("Enter amount: "))

    category = input("Enter category: ")



    expenses.append((name, amount, category))



    choice = input("Add more? (y/n): ")

    if choice == 'n':

        break



print("Expenses:", expenses)
```

Output:

```
Enter expense name: food
Enter amount: 100
Enter category: grocieries
Add more? (y/n): y
Enter expense name: transport
Enter amount: 50
Enter category: travel
Add more? (y/n): n
Expenses: [('food', 100.0, 'grocieries'), ('transport', 50.0, 'travel')]
```

Improved Prompt:

Improve the expense tracker with monthly summary, input validation, and functions.

Improved Code:

```
def add_expense(expenses):

    name = input("Enter expense name: ")



    while True:

        try:

            amount = float(input("Enter amount: "))

            break

        except:

            print("Invalid amount")



    category = input("Enter category: ")

    expenses.append({"name": name, "amount": amount, "category":
category})




def show_summary(expenses):

    total = sum(e["amount"] for e in expenses)
```

```python
    print("Total Monthly Expense:", total)




expenses = []



while True:

    add_expense(expenses)

    choice = input("Add more? (y/n): ")

    if choice == 'n':

        break



show_summary(expenses)
```

Output:

```
Enter expense name: food
Enter amount: 100
Enter category: grocieries
Add more? (y/n): y
Enter expense name: transport
Enter amount: 50
Enter category: travel
Add more? (y/n): n
Total Monthly Expense: 150.0
```

Explanation:

Functions → modular design
Validation → avoids runtime errors
Summary → real-world usability

Task 2: Git Version Control (Student Grade Calculator)

Initialize Repo,Initial Code,Create Branch

```
av-750XGK:~/Downloads/assistantcoding-20260312T084629Z-3-001$ cd /home/abhinav/Downloads/grade-calculator
d. && git commit -m "Iinitial grade calculator with average"
 -global to set the identity only in this repository.

unable to auto-detect email address (got 'abhinav@750XXGK.%.(none)')
 config user.name "abhinav" && git config user.name "abhinav" && " conig user.email "abhinav" &&&& git config user.ema
 (root-commit) 15776b) Initial grade
 changed, 3 insertions(+)
mode 100644 grade_galde_calculator.py

v@bhinav-750XGK:~/Downloads/assistantcoding- cd /home/abhinav/Downloads/grabhinav && git checkout -b feature-ai-grade &
me/abhinav/Downloads/grade-calculator &&
av@bhinav-750XGK:~/Downloads/grade-calculator$ cd /home/abhinav/Download/Downloads/grade-calculator && git + grade_ca

ks = list(map(int, input("Enter marks: ").split()))
al = sum(marks)
 = total / len(marks)
centage = (total / (len(marks)*100)) * 100

avg >= 90:
 grade = 'A'
if avg >= 80:
 grade = 'B'
if avg >= 70:
 grade = 'C'
se:
 grade = 'F'

tt("Aerage:", avg)
nt("Percentage",percentage)
nt("Grade", grade)

av@bhinav-7500XGK:~/Downloads/grade-ccalculator$
av@bhinav-750XGK:~~/Downloads/grade-calculatorr$ git add .&& git commit -m "Added grade classification and percentage cal
```

AI Prompt:

Add grade classification and percentage calculation to the program.

Updated Code:

```
marks = list(map(int, input("Enter marks: ").split()))



total = sum(marks)

avg = total / len(marks)

percentage = (total / (len(marks)*100)) * 100



if avg >= 90:

    grade = 'A'

elif avg >= 75:

    grade = 'B'
```

```
elif avg >= 50:

    grade = 'C'

else:

    grade = 'F'



print("Average:", avg)

print("Percentage:", percentage)

print("Grade:", grade)
```

Commit + merge:

```
abhinav@abhinav-750XGK:~/Downloads/quiz-management$ git checkout -b feature-score
Switched to a new branch 'feature-score'

abhinav@abhinav-750XGK:~/Downloads/quiz-management$git add .

abhinav@abhinav-750XGK:~/Downloads/quiz-management$ git commit -m "Addded score calculation"
[feature-score 8ab91cd] Added score calculation
 1 file changed, 12 insertions(+), 1 deletion(-)

abhinav@abhinav-750XGK:~/Downloads/quiz-management$ git checkout main
Switched to branch 'main'
8ab91cd | 1 ++++++++-
 1 file changed, 11 inserions-1, deletion(-)

abhinav@abhinav-750XGK:~/Downloads/quiz-management$ git merge feature-score
Capital of India? Delhi
2+2 4
Correct
Final Score: 1

abhinav@abhinav-750XGK:~/Downloads/quiz-management$ git log --oneline --decorate --graph --all --max-count=5
* 8ab91cd (HEAD => main, feature-score)
 Added score calculation *****$
* 2f4a7d1 Initial quiz management system

abhinav@abhinav-750XGK:~/Downloads$
```

Justification:

Branching safe feature development, Merge integration workflow, AI faster feature addition.

Task 3: Modify Existing Resource Using AI Assistance

Setup Code:

```
import re



password = input("Enter password: ")



if len(password) < 8:

   print("Weak password")

elif not re.search(r'[!@#$%^&*]', password):

   print("Add special character")

else:

   print("Strong password")
```

Prompt:

Generate a commit message for adding password validation with length and special character rules.

AI Committed Message:

```
-750XGK:~/Downloads/aiasistantcoding-20260312T084629Z-3-001$
jav-750XGK:~/Downloads/aiassistantcoding-20260312T084629Z-3-001$ git comm
t -m "feat: add password validation with minimum length and special charac
```

Human Version:

Added strong password validation rules

Justification:

AI more structured (uses “feat:” convention), Human → less descriptive

Task 4: Quiz Management System (Pair Programming)

Student A Code:

```
questions = {
   "Capital of India?": "Delhi",
   "2+2?": "4"
}


for q in questions:
   ans = input(q + " ")
   if ans == questions[q]:
       print("Correct")
   else:
      print("Wrong")
```

AI Prompt (Student B):

Add score calculation and input validation to the quiz program.

Final Code:

```
questions = {
   "Capital of India?": "Delhi",
   "2+2?": "4"
}


score = 0
```

```python
for q in questions:
    ans = input(q + " ").strip()

    if ans.lower() == questions[q].lower():
        print("Correct")
        score += 1
    else:
        print("Wrong")

print("Final Score:", score)
```

Git WorkFlow:

```
shashidhar@shashidhar-750XGK:~/Downloads/quiz-management$ git checkout -b feature-score
Switched to a new branch 'feature-score'

shashidhar@shashidhar-750XGK:~/Downloads/quiz-management$ git add .

shashidhar@shashidhar-750XGK:~/Downloads/quiz-management$ git commit -m "Added score calculation"
[feature-score 8ab91cd] Added score calculation
 1 file changed, 12 insertions(+), 1 deletion(-)

shashidhar@shashidhar-750XGK:~/Downloads/quiz-management$ git checkout main
Switched to branch 'main'

shashidhar@shashidhar-750XGK:~/Downloads/quiz-management$ git merge feature-score
Updating 2f4a7d1..8ab91cd
Fast-forward
 quiz.py | 12 +++++++++++-
 1 file changed, 11 insertions(+), 1 deletion(-)

shashidhar@shashidhar-750XGK:~/Downloads/quiz-management$ python quiz.py
Capital of India? Delhi
Correct
2+2? 5
Wrong
Final Score: 1

shashidhar@shashidhar-750XGK:~/Downloads/quiz-management$ git log --oneline --decorate --graph --all --max-count=5
* 8ab91cd (HEAD -> main, feature-score) Added score calculation
* 2f4a7d1 Initial quiz management system
```

Output:

```
abhinav@abhinav-750XGK:~/Downloads$ /usr/bin/python3 /home/abhinav/Downloads/aiassistantcoding-20260312T084629Z-3-001/aiassistantcoding/65.py
Capital of India? y
Wrong
2+2? 4
Correct
Final Score: 1
abhinav@abhinav-750XGK:~/Downloads$
```

Justification:

- Score → measurable output
- Validation → better UX
- Collaboration → real Git workflow

Task 5: AI-Assisted API Documentation and Readability

Prompt:

Add docstrings and comments to Flask API endpoints and optionally integrate Swagger documentation.

Code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    """
    GET /status
    Returns the API status.

    Responses:
        200: {'status': 'ok', 'message': 'API is running'}
    """
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    """
    POST /requests
    Creates a new service request.

    Request Body (JSON):
        {'title': 'string', 'priority': 'string'}

    Responses:
        201: New request object with 'id', 'title', 'priority'
        400: {'error': 'Missing title or priority'}
    """
    global request_id_counter
    data = request.get_json()
    # Validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    # Create a new request object
    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    """
    GET /requests
    Retrieves all service requests.

    Responses:
        200: A list of all request objects.
    """
    return jsonify(requests)

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    """
    PUT /requests/<request_id>
    Updates an existing service request by its ID.

    Path Parameters:
        request_id (int): The ID of the request to update.
```

```
    # Validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    # Find and update the request
    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    '''
    DELETE /requests/<request_id>
    Deletes a service request by its ID.

    Path Parameters:
        request_id (int): The ID of the request to delete.

    Responses:
        200: {'message': 'Request with id {request_id} deleted successfully'}
        404: {'error': 'Request not found'}
    '''
    global requests
    original_length = len(requests)
    # Filter out the request to be deleted
    requests = [req for req in requests if req['id'] != request_id]
    if len(requests) < original_length:
        return jsonify({'message': f'Request with id {request_id} deleted successfully'}), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

Explanation:

Docstrings → improve readability
Swagger → auto API documentation. Helps other developers integrate faster